

Flannel

Flannel이란?

Flannel은 CoreOS에서 개발한 쿠버네티스용 CNI 플러그인입니다. 쿠버네티스 클러스터에서는 각 노드에 존재하는 파드들이 서로 다른 노드에 있는 파드와도 통신할 수 있어야 하는데, Flannel은 이 문제를 해결하기 위한 가장 기본적이고 단순한 솔루션으로 알려져 있습니다.

Flannel은 CNI 중에서 매우 단순하고 간단한 방법으로 동작합니다. 따라서 설치와 설정이 매우 간단합니다. 별도의 복잡한 컨트롤 플레인 구성 없이 DaemonSet 형태로 각 노드에 **flanneld** 프로세스를 배포하기만 하면 됩니다. 또한 BGP 같은 복잡한 라우팅 프로토콜 없이도 각 노드 간 서브넷 정보를 공유하여 라우팅을 구성할 수 있습니다.

다만, 간단한 만큼 단점도 있습니다. Flannel은 **NetworkPolicy** 기능을 지원하지 않습니다. 따라서 순순하게 파드 간 통신 목적으로는 괜찮지만, 복잡한 네트워크 정책을 설정하는 데에는 적합하지 않습니다.

하지만, 단점에도 불구하고 단순함 덕분에 Flannel은 쿠버네티스 초기부터 지금까지도 가장 널리 사용되는 CNI 플러그인 중 하나로 자리잡고 있습니다. 다만 세밀한 트래픽 제어가 필요한 환경에서는 Calico나 Cilium 같은 다른 CNI를 사용합니다.

Flannel 동작 방식

Flannel은 **Overlay Network**를 통해 Pod간 통신을 지원합니다. Overlay Network가 무엇인지 먼저 알아보겠습니다.

An **overlay network** is a logical computer network that is layered on top of a physical network. - Wikipedia

Wikipedia에서는, Overlay Network는 Physical Network 위에 생성된 가상 네트워크라고 정의하고 있습니다. Overlay Network를 통해 클러스터 내 모든 Pod가 같은 가상 네트워크에 위치하게 되고, 실제 물리 네트워크 인프라와 무관하게 파드들이 마치 하나의 평평한 네트워크에 있는 것처럼 통신할 수 있게만 들어주는 것이 오버레이 네트워크의 역할입니다.

평평한 네트워크를 만드는 것은 이해했습니다. 그런데 가상 네트워크를 사용하더라도 결국에는 물리 네트워크를 통해 이동해야 할텐데, 이 부분은 어떻게 동작하는 거죠?

Flannel이 설치되면 Flanneld라는 데몬 프로세스가 생성됩니다. flanneld는 라우팅 테이블에 새로운 라우팅 규칙을 생성합니다. 예를 들어, 클러스터의 IP CIDR 대역이 `100.96.0.0/16` 이라고 가정해보겠습니다. 그렇다면 라우팅 테이블은 다음과 같이 구성됩니다.

```
$ ip route
default via 172.20.32.1 dev eth0
100.96.0.0/16 dev flannel0 proto kernel scope link src 100.96.1.0
172.20.32.0/19 dev eth0 proto kernel scope link src 172.20.33.102
```

Node A에 있는 Pod A(100.96.1.1)가 Node B에 있는 Pod B(100.96.2.2)로 패킷을 전송하고 싶다고 해보겠습니다. 목적지인 100.96.2.2 는 100.96.0.0/16 대역에 포함되므로, 라우팅 테이블 두 번째 줄의 규칙에 따라 **flannel0**이라는 인터페이스로 이동하게 됩니다.

여기서 flannel0이라는 이름을 처음 보면 물리적인 네트워크 카드처럼 느껴질 수 있는데, 사실 flannel0은 실제 하드웨어가 아니라 리눅스 커널이 제공하는 가상의 네트워크 인터페이스인 **TUN 디바이스**입니다.

TUN 디바이스는 물리 네트워크 카드가 하는 일을 소프트웨어적으로 흉내 내는 장치로, Kernel과 User Space 프로세스 사이에서 IP 패킷을 서로 주고받을 수 있게 해주는 통로 역할을 합니다. 커널이 이 장치로 패킷을 내려보내면 사용자 공간에서 실행 중인 애플리케이션이 그 패킷을 읽어갈 수 있고, 반대로 사용자 공간 애플리케이션이 이 장치에 패킷을 써넣으면 커널은 그 패킷이 마치 실제 네트워크 카드로 들어온 것처럼 처리합니다.

패킷이 flannel0으로 라우팅되면, Flannel 데몬 프로세스(flanneld)가 이를 받습니다. 그리고 flanneld는 목적지를 확인하고, 패킷을 감싼 후 다시 전송합니다.

근데 목적지 Pod가 있는 노드의 실제 물리 주소는 어떻게 알죠? Node A의 flanneld는 100.96.2.2라는 IP가 Node B에 있다는 사실을 어떻게 알 수 있을까요?

이 정보는 **etcd**에 저장되어 있습니다. Flannel이 처음 클러스터에 설치되고 새로운 노드가 조인할 때마다, 그 노드의 flanneld는 etcd로부터 아직 다른 노드가 사용하지 않은 서브넷 대역을 하나 할당받습니다. 예를 들어 Node A는 100.96.1.0/24를, Node B는 100.96.2.0/24를 할당받는 식입니다.

이렇게 어떤 서브넷 대역이 어떤 노드의 실제 물리 IP에 속해 있는지에 대한 매핑 정보가 다시 etcd에 기록되고, 클러스터에 있는 모든 flanneld 프로세스는 이 정보를 지속적으로 감시합니다. 그래서 Node A의 flanneld는 패킷의 목적지 IP인 100.96.2.2가 100.96.2.0/24 대역에 속한다는 것, 그리고 이 대역이 Node B의 실제 물리 IP(예: 172.20.33.103)에 매핑되어 있다는 것을 etcd에 저장된 정보를 통해 알 수 있는 것입니다.

정리하면, Flannel은 각 노드마다 flanneld 라는 데몬 프로세스를 생성하고, etcd에 저장된 호스트-서브넷 매핑 정보를 바탕으로 라우팅 테이블을 업데이트하여 목적지 대역의 패킷이 flannel0 TUN 디바이스로 전달받도록 만들고, flanneld가 이 패킷을 넘겨받아 캡슐화하여 목적지 노드의 실제 물리 IP로 재전송하는 방식입니다.

UDP 방식

앞서 설명한 흐름에서, flanneld가 TUN 디바이스로부터 넘겨받은 패킷을 감싸서 전달한다고 했습니다. 패킷을 감싸는 방법에도 여러가지가 있을 수 있지만 Flannel이 사용하는 가장 기본적인 방법이 UDP 패킷을 이용한 캡슐화 방식입니다.

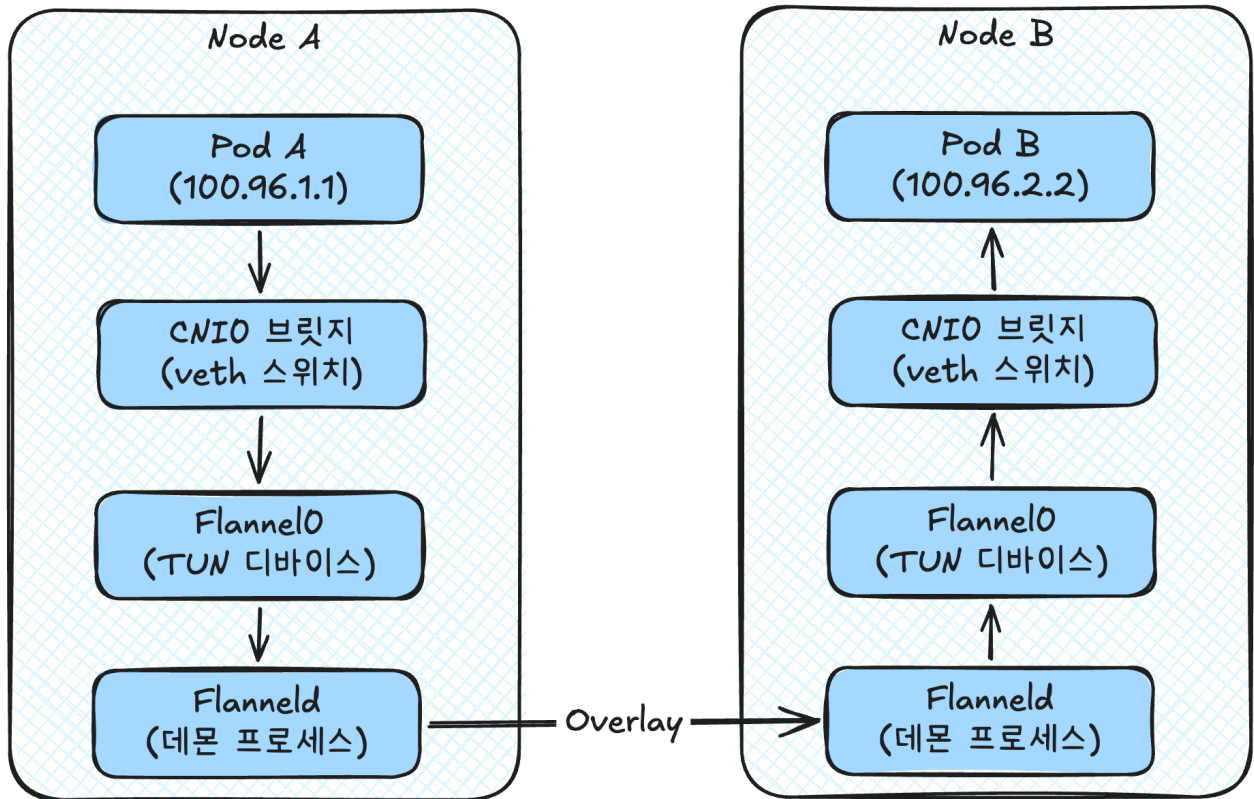
이 방식의 핵심 아이디어는 단순합니다. 목적지 파드의 가상 IP를 담은 원본 IP 패킷 전체를, 실제 물리 노드끼리 주고받을 수 있는 UDP 패킷의 payload에 통째로 집어넣어 버리는 것입니다. 원본 IP 패킷의 수신자 주소(목적지 Pod IP)는 가상 네트워크에서만 통용되는 주소이기 때문에 실제 물리 네트워크는 이 주소를 읽을 수 없지만, flanneld가 새롭게 감싼 UDP 패킷에는 물리 네트워크가 이해할 수 있는 실제 물리 노드의 IP 주소가 적혀 있어서 목적지 노드까지 무사히 전달될 수 있습니다.

이 과정을 조금 더 순서대로 풀어보면 다음과 같습니다.

1. 파드에서 다른 노드에 있는 파드로 패킷을 보내면, 이 패킷은 먼저 로컬 브릿지(cni0)를 거쳐 flannel0 TUN 디바이스로 전달됩니다.
2. 커널은 이 IP 패킷을 사용자 공간에서 실행 중인 flanneld 프로세스로 넘겨줍니다.
3. flanneld는 사용자 공간에서 원본 IP 패킷 전체를 UDP 패킷의 payload에 담아 캡슐화합니다.
4. 캡슐화된 UDP 패킷은 다시 커널로 전달되어, 목적지 노드의 실제 물리 IP 주소로 전송됩니다.
5. 목적지 노드에서는 반대의 과정을 거칩니다. UDP 패킷을 받은 flanneld가 사용자 공간에서 UDP 헤더를 제거(디캡슐화)하여 원본 IP 패킷을 복원하고, 이를 다시 TUN 디바이스를 통해 커널로 전달합니다.
6. 커널은 복원된 원본 패킷을 로컬 브릿지를 거쳐 목적지 파드로 전달합니다.

정리하면, Flannel의 IP-IP 방식은 캡슐화와 디캡슐화 방식으로 노드 간에 패킷 전송을 가능하게 합니다. 그리고 이 캡슐화 및 디캡슐화 작업은 flanneld라는 데몬 프로세스에서 이루어집니다.

다이어그램



UDP 방식을 권장하지 않는 이유 (성능)

지금까지 설명한 UDP 캡슐화 방식은 구조가 단순하다는 장점이 있지만, 실제 운영 환경에서는 성능 문제로 인해 권장되지 않는다고 합니다.

일반적으로 애플리케이션이 네트워크 데이터를 주고받을 때, 데이터는 사용자 공간(User Mode)과 커널 공간(Kernel Mode) 사이의 경계를 넘나들어야 합니다. 그리고 이 경계를 넘을 때마다 Context Switching 이 발생합니다. Context Switching 은 CPU가 지금까지 실행하고 있던 작업의 상태(레지스터 값, 스택 포인터, 메모리 매핑 정보 등)를 저장해두고, 다른 작업의 상태를 다시 불러오는 과정입니다. 이 과정 자체가 CPU 사이클을 소모하는 무거운 작업이며, 동시에 데이터가 커널 버퍼와 사용자 공간 버퍼 사이를 오갈 때마다 메모리 복사도 함께 일어납니다.

그런데 UDP 캡슐화 방식으로 Pod A(100.96.1.1)에서 Pod B(100.96.2.2)로 패킷 하나가 이동하는 전체 과정을 되짚어보면, 이 경계를 몇 번이나 넘게 되는지 세어볼 수 있습니다.

1. **User Mode → Kernel Mode:** Pod A(사용자 공간의 컨테이너 프로세스)에서 생성된 패킷이 커널 네트워크 스택으로 전달됩니다.
2. **Kernel Mode → User Mode:** 커널은 이 패킷을 flannelO TUN 디바이스를 통해 다시 사용자 공간에서 실행되고 있는 flanneld 프로세스로 넘깁니다. flanneld가 UDP 캡슐화를 수행하는 곳이 바로 사용자 공간이기 때문입니다.
3. **User Mode → Kernel Mode:** flanneld가 캡슐화를 마친 UDP 패킷을 다시 커널로 전달하여, 실제 물리 네트워크(eth0)를 통해 Node B로 전송합니다.

즉 패킷 하나를 전송하는 데에만 3번의 모드 전환이 필요합니다. 만약 트래픽이 많지 않고 패킷 수가 적다면 이 오버헤드는 크게 체감되지 않을 수 있습니다. 하지만 실제 서비스 환경에서는 초당 수천, 수만 건의 패킷이 오가는 것이 일반적이고, 이 모든 패킷 하나하나가 매번 이 세 번의 전환 과정을 거쳐야 합니다. 그 결과 CPU 사용률이 증가하고, 패킷이 목적지에 도착하기까지 걸리는 지연 시간도 함께 늘어날 수 있습니다.

이런 이유로 Flannel의 UDP 백엔드 방식은 오늘날 실제 운영 환경에서는 거의 사용되지 않고, 커널이 VXLAN 같은 캡슐화 기능을 아예 지원하지 않는 아주 특수한 예외 상황을 위한 최후의 대체 수단으로만 남아 있습니다. 대신 실제로 널리 권장되는 방식은 VXLAN 백엔드와 host-gw 백엔드입니다. VXLAN은 캡슐화 작업 자체를 커널 안에서 처리하기 때문에 사용자 공간으로 패킷을 끌어올릴 필요가 없고, host-gw는 애초에 캡슐화라는 과정 자체를 생략하고 커널 라우팅 테이블만으로 패킷을 노드 간에 직접 전달합니다. 두 방식 모두 사용자 공간의 개입 없이 커널 안에서 패킷 처리가 끝나기 때문에, 앞서 설명한 모드 전환 오버헤드가 발생하지 않아 UDP 방식보다 훨씬 뛰어난 성능을 보여줍니다.

VxLAN

VxLAN은 L3 네트워크 위에서 L2 네트워크를 구현한 기술이라고 생각하시면 됩니다. Flannel에서 VxLAN을 사용하게 되면, 아래의 작업들이 실행됩니다.

- **vxlan 디바이스 생성:** flanneld는 노드가 시작될 때 `flannel.1`이라는 vxlan 타입의 네트워크 디바이스를 커널에 만들어 달라고 요청합니다(내부적으로는 netlink를 통해 `ip link add flannel.1 type vxlan ...`에 해당하는 작업을 수행합니다).
- **다른 노드 정보를 라우팅 테이블·ARP·FDB에 등록:** 새로운 노드가 클러스터에 조인하면, flanneld는 그 노드에 대해 세 가지 항목을 커널에 등록합니다.
 - **라우팅 테이블 항목:** "이 노드의 서브넷 대역은 flannel.1 디바이스를 통해, 이 노드를 next-hop으로 해서 보내라."
 - **ARP 항목:** 상대 노드의 flannel.1 디바이스(VTEP)에 대한 정적 MAC 주소 정보.
 - **FDB(Forwarding Database) 항목:** 그 VTEP MAC 주소가 실제로는 어떤 물리 IP로 찾아가야 하는지에 대한 매핑.

이 세 항목이 한 번 등록되고 나면, 이후 실제 패킷이 오갈 때는 커널이 라우팅 테이블 → ARP 테이블 → FDB 테이블을 순서대로 조회해서 스스로 캡슐화 대상과 목적지를 결정합니다. UDP 방식과 다르게 User Space를 왔다갔다 하지 않고 커널에서 모두 처리되기 때문에 성능의 이점이 있습니다.

라우팅 테이블은 왜 필요한거죠?

파드 입장에서 패킷을 보낼 때, 목적지 IP가 우리 노드 안에 있는 파드인지, 아니면 다른 노드에 있는 파드인지를 가장 먼저 판단해야 합니다. 이 판단 기준이 바로 라우팅 테이블입니다. 노드 A의 라우팅 테이블에는 "노드 B가 할당받은 서브넷 대역(예: `100.96.2.0/24`)으로 가는 패킷은 flannel.1 디바이스를 통해, next-hop은 노드 B로 보내라"는 규칙이 들어 있습니다. 만약 이 규칙이 없다면, 커널은 `100.96.2.2`라는 목적지 IP가 로컬 네트워크에 없다는 사실만 알 뿐, 어느 인터페이스로 내보내야 하는지 전혀 알 수 없습니다.

따라서 정리하면, 라우팅 테이블은 가상 IP 목적지를 어떤 인터페이스(flannel.1)로 내보낼지를 결정하기 위해 필요합니다.

ARP는 왜 필요한거죠?

라우팅 테이블이 flannel.1을 통해, next-hop까지 알려줬다고 해도, 이더넷 프레임을 만들기 위해서는 IP 주소가 아니라 MAC 주소가 필요합니다. 이더넷 계층은 IP를 몰라도 동작할 수 있지만, MAC 주소 없이는 프레임 자체를 만들 수 없기 때문입니다. 이 "next-hop IP → MAC 주소" 변환을 담당하는 것이 ARP입니다. 다만 일반적인 물리 네트워크에서는 ARP 요청을 브로드캐스트로 뿌려서 응답을 기다려야 하지만, Flannel의 VXLAN 백엔드에서는 flanneld가 노드가 조인할 때 이미 상대 노드의 flannel.1 MAC 주소를 알고 있는 상태이므로, 이 값을 미리 정적(static) ARP 항목으로 커널에 등록해둡니다. 그래서 실제 통신 시점에는 브로드캐스트나 대기 없이 이미 등록된 값을 즉시 사용할 수 있습니다.

FDB는 왜 필요한거죠?

ARP를 통해 목적지 MAC 주소까지는 알아냈지만, 이 MAC 주소는 어디까지나 가상 네트워크 안에서만 통용되는 주소입니다. 물리 네트워크 장비는 이 MAC 주소를 전혀 알지 못합니다. 그래서 "이 가상 MAC 주소를 가진 장비가 실제로는 어느 물리 노드(물리 IP)에 있는지"를 다시 한번 변환해줘야 합니다. 일반적인 이더넷 스위치가 "MAC 주소 → 물리 포트 번호"를 기록하는 것과 유사하게, VXLAN의 FDB는 "(VNI, MAC 주소) → 실제 물리 IP"를 기록합니다. 이 정보가 있어야 vxlan 커널 모듈이 캡슐화된 패킷의 Outer IP 헤더에 어떤 목적지 IP를 적어야 할지 최종적으로 결정할 수 있습니다.

